



K.R. Mangalam University

School of Engineering & Technology

DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

Program: BCA (AI & DS)

Course Code - ENCA301 (2025-2026)

Submitted by:

Name: Bhavya Rattan

Roll Number: 2401201004

Course: BCA (AI & DS) - Section B

Submitted To:

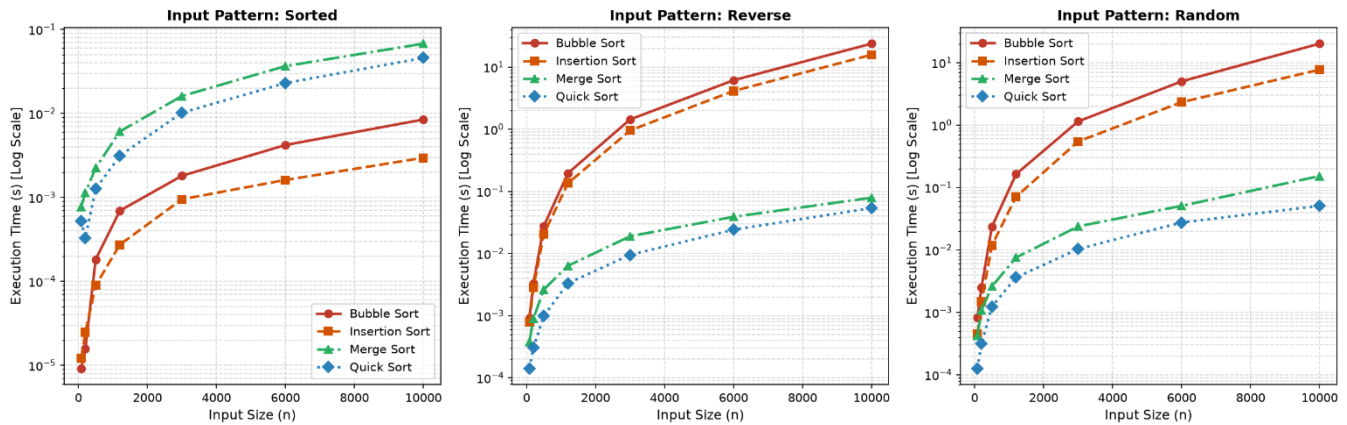
Dr. Aarti Sangwan

Assistant Professor

6. Visualizations and Observations

6.1 Sorting Algorithm Execution Time vs Input Size

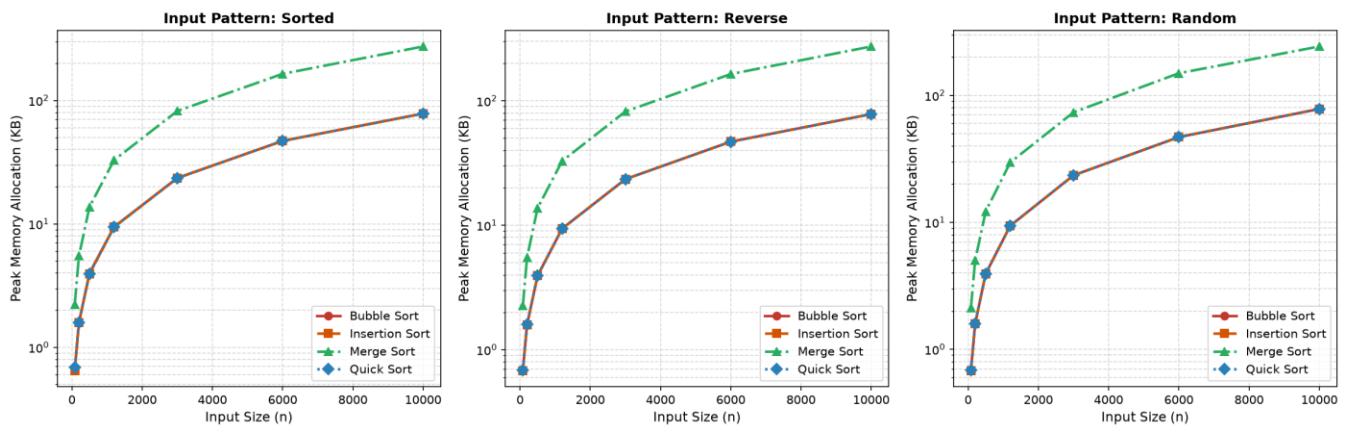
Sorting Algorithm Performance: Execution Time vs Input Size



Observations: The graph compares the execution time of Bubble Sort, Insertion Sort, Merge Sort, and Quick Sort for sorted, reverse-sorted, and random input patterns. Bubble Sort and Insertion Sort show a much steeper increase in execution time as the input size grows, particularly for reverse and random inputs, reflecting their $O(n^2)$ complexity. In contrast, Merge Sort and Quick Sort scale more efficiently with approximately $O(n \log n)$ behaviour. Overall, Quick Sort provides the fastest performance for large inputs in the experiment, while Bubble Sort shows the slowest growth in performance.

6.2: Sorting Algorithm Resource Requirements – Peak Memory Usage

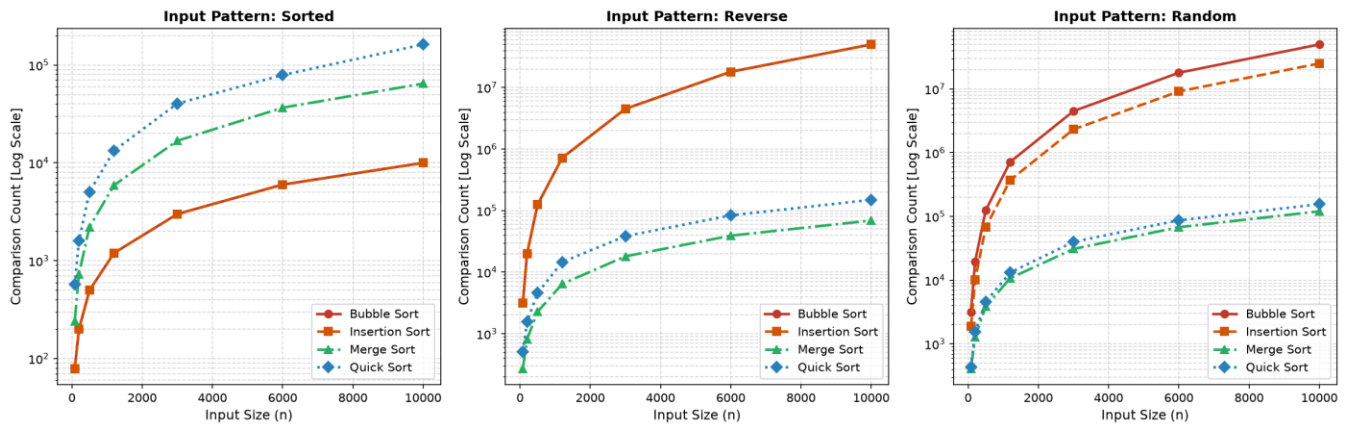
Sorting Algorithm Resource Requirements: Peak Memory Usage



Observations: The graph compares the peak memory allocation of the sorting algorithms for sorted, reverse, and random input patterns. Bubble Sort, Insertion Sort, and Quick Sort show similar memory usage, increasing gradually with input size. In contrast, Merge Sort requires significantly more memory because it creates additional arrays during the merging process. This behaviour supports its theoretical $O(n)$ auxiliary space complexity, while the other algorithms use

6.3: Sorting Algorithm Operation Counts – Key Comparisons

Sorting Algorithm Operation Counts: Key Comparisons

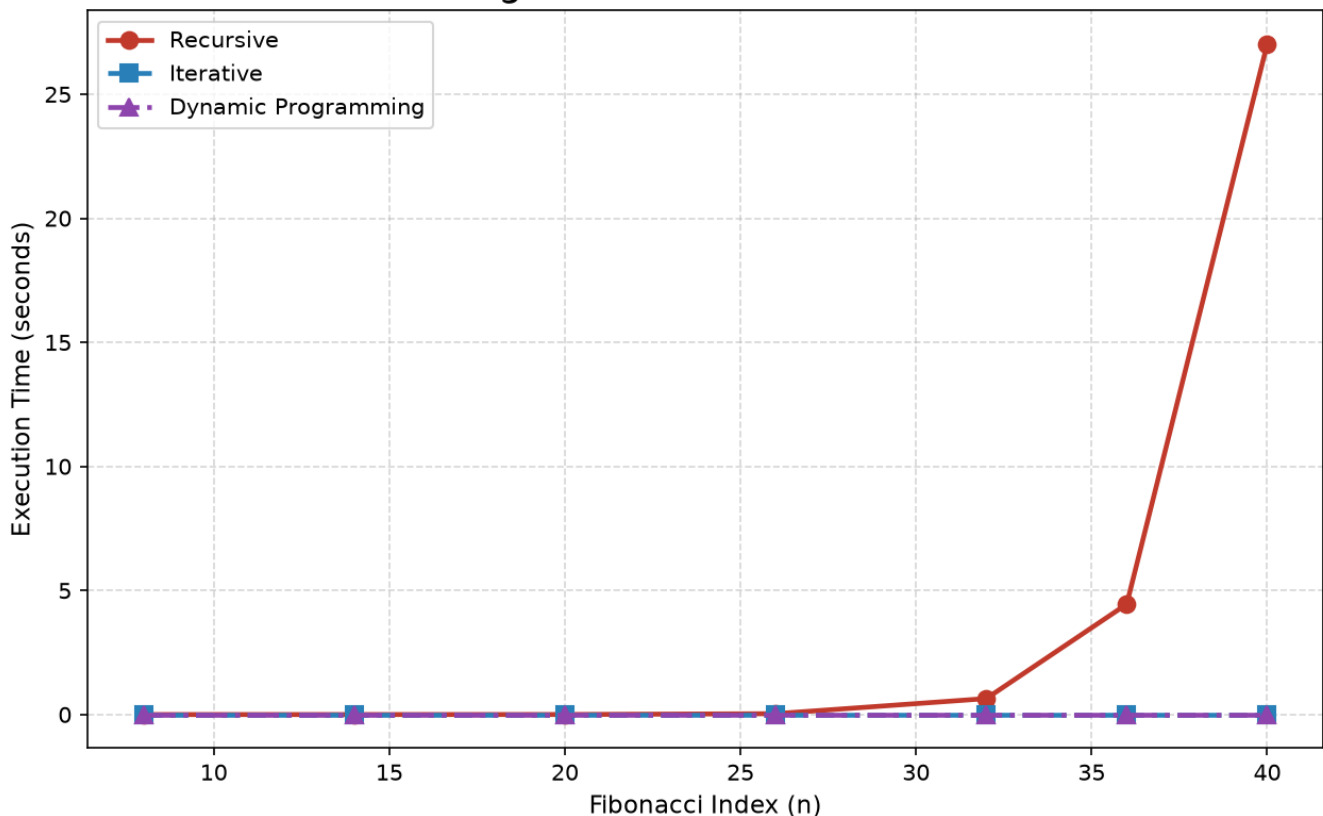


Figure

Observations: The graph shows the number of comparisons performed by each sorting algorithm for different input patterns. Bubble Sort and Insertion Sort require significantly more comparisons as the input size increases, especially for reverse and random inputs, reflecting their $O(n^2)$ complexity. Merge Sort and Quick Sort perform considerably fewer comparisons, showing better scalability with approximately $O(n \log n)$ behaviour. Overall, the graph demonstrates why Merge Sort and Quick Sort are more efficient for larger datasets..

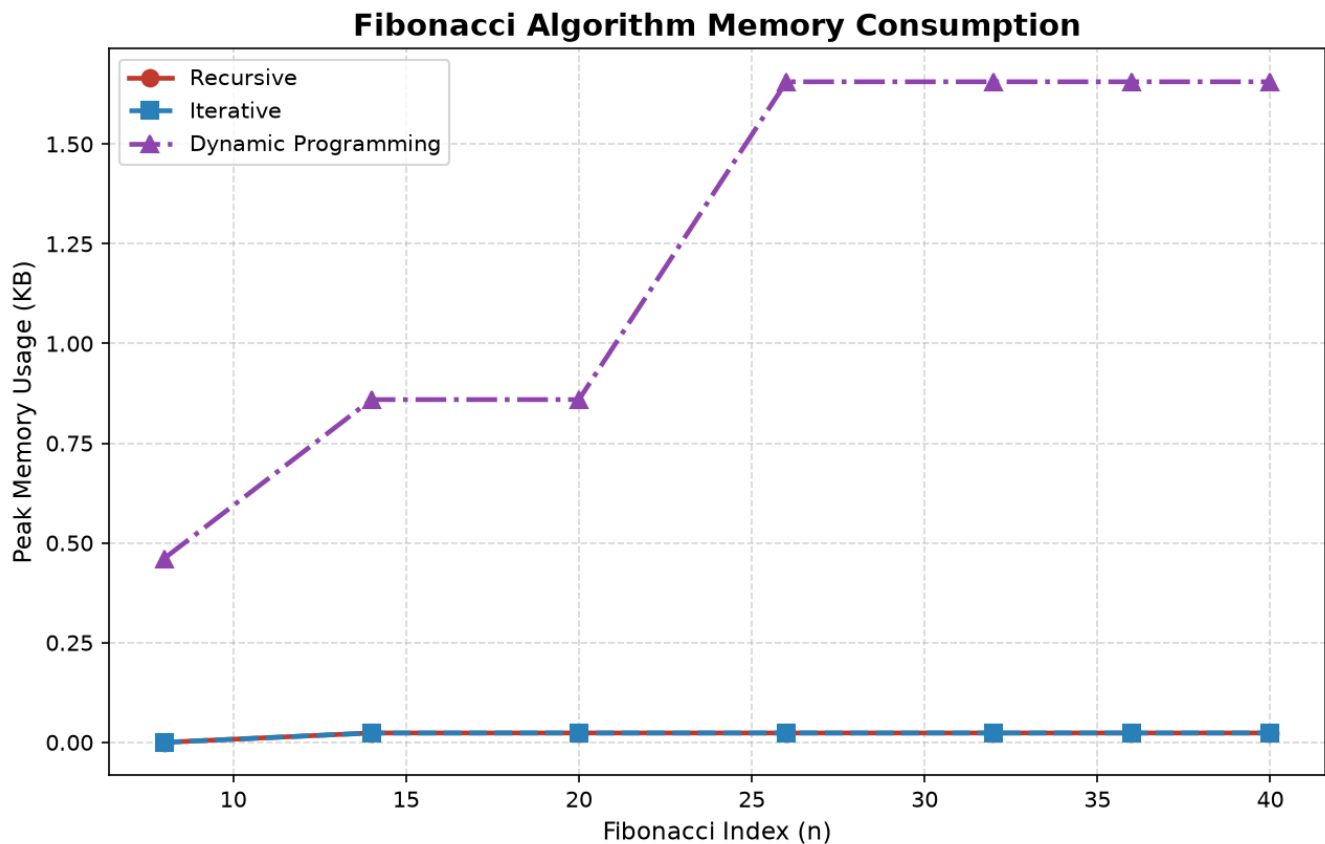
6.4: Fibonacci Algorithm Execution Time Benchmark

Fibonacci Algorithm Execution Time Benchmark



Observations: The graph shows that the Recursive Fibonacci algorithm becomes significantly slower as the Fibonacci index increases, reaching approximately 27 seconds at $n = 40$. In contrast, the Iterative and Dynamic Programming approaches maintain extremely low execution times throughout the experiment. This clearly demonstrates the poor scalability of naive recursion compared to the more efficient $O(n)$ approaches

6.5: Fibonacci Algorithm Memory Consumption



Figure

Observations: The graph compares the peak memory usage of the three Fibonacci algorithms. The Iterative and Recursive approaches maintain very low memory usage, while the Dynamic Programming approach uses more memory as the input size increases because it stores previously calculated values. This demonstrates the $O(n)$ space requirement of Dynamic Programming compared with the $O(1)$ space efficiency of the Iterative approach.

7. Discussion

7.1 Theoretical vs Experimental Performance

The experimental results demonstrate the practical importance of algorithm complexity. The sorting experiments showed that Bubble Sort and Insertion Sort become significantly slower as the input size increases due to their $O(n^2)$ time complexity. Among them, Insertion Sort performed better than Bubble Sort for random input data.

Merge Sort and Quick Sort showed much better scalability for larger datasets. Quick Sort achieved the fastest execution time in the experiments, while Merge Sort provided consistent performance across different input patterns. However, Merge Sort required more memory because of its additional merging operations.

The Fibonacci experiments showed an even greater difference in algorithm efficiency. The Naive Recursive approach experienced a rapid increase in execution time and required approximately 27 seconds at $n = 40$. In contrast, the Iterative and Dynamic Programming approaches maintained extremely low execution times. The Iterative approach also used the least memory, while Dynamic Programming required additional memory to store previously calculated values.

Overall, the experimental results were consistent with the theoretical complexity analysis and demonstrated how algorithm selection becomes increasingly important for larger input sizes.

8. Conclusion

This study compared the performance of different sorting and Fibonacci algorithms using execution time, comparison count, and memory consumption.

The results showed that Quick Sort and Merge Sort are more suitable for large datasets because of their efficient $O(n \log n)$ behaviour. Quick Sort provided the fastest execution time in the recorded experiments, while Merge Sort provided stable and consistent performance at the cost of higher memory usage.

For Fibonacci computation, the Iterative approach provided the best overall performance because it achieved $O(n)$ time complexity with $O(1)$ space complexity. The Naive Recursive approach was found to be inefficient for larger values of n because of its exponential growth.

Therefore, the experiments successfully demonstrate the relationship between theoretical algorithm complexity and practical performance, highlighting the importance of selecting appropriate algorithms based on input size and resource requirements.